

File: paging-linear-translate.py

```
#!/usr/bin/env python
```

```
import sys
from optparse import OptionParser, SUPPRESS_HELP
import random
import math
```

```
def mustbepowerof2(bits, size, msg):
    if math.pow(2,bits) != size:
        print &#x27;Error in argument: %s&#x27;; % msg
        sys.exit(1)
```

```
def mustbemultipleof(bignum, num, msg):
    if (int(float(bignum)/float(num)) != (int(bignum) / int(num))):
        print &#x27;Error in argument: %s&#x27;; % msg
        sys.exit(1)
```

```
def convert(size):
    length = len(size)
    lastchar = size[length-1]
    if (lastchar == &#x27;k&#x27;) or (lastchar == &#x27;K&#x27;):
        m = 1024
        nsize = int(size[0:length-1]) * m
    elif (lastchar == &#x27;m&#x27;) or (lastchar == &#x27;M&#x27;):
        m = 1024*1024
        nsize = int(size[0:length-1]) * m
    elif (lastchar == &#x27;g&#x27;) or (lastchar == &#x27;G&#x27;):
        m = 1024*1024*1024
        nsize = int(size[0:length-1]) * m
    else:
        nsize = int(size)
    return nsize
```

```
#
# main program
#
```

```
parser = OptionParser()
parser.add_option(&#x27;-A&#x27;, &#x27;--addresses&#x27;, default=&#x27;-1&#x27;,
                help=&#x27;a set of comma-separated pages to access; -1 means randomly generate&#x27;,
                action=&#x27;store&#x27;, type=&#x27;string&#x27;, dest=&#x27;addresses&#x27;)
parser.add_option(&#x27;-s&#x27;, &#x27;--seed&#x27;, default=0, help=&#x27;the random seed&#x27;,
                action=&#x27;store&#x27;, type=&#x27;int&#x27;, dest=&#x27;seed&#x27;)
parser.add_option(&#x27;-a&#x27;, &#x27;--asize&#x27;, default=&#x27;16k&#x27;, help=&#x27;address space size&#x27;,
                action=&#x27;store&#x27;, type=&#x27;string&#x27;, dest=&#x27;asize&#x27;)
parser.add_option(&#x27;-p&#x27;, &#x27;--physmem&#x27;, default=&#x27;64k&#x27;, help=&#x27;physical memory size&#x27;,
                action=&#x27;store&#x27;, type=&#x27;string&#x27;, dest=&#x27;psize&#x27;)
parser.add_option(&#x27;-P&#x27;, &#x27;--pagesize&#x27;, default=&#x27;4k&#x27;, help=&#x27;page size (e.g., 4k)&#x27;,
                action=&#x27;store&#x27;, type=&#x27;string&#x27;, dest=&#x27;pagesize&#x27;)
parser.add_option(&#x27;-n&#x27;, &#x27;--numaddrs&#x27;, default=5, help=&#x27;number of virtual addresses to use&#x27;,
                action=&#x27;store&#x27;, type=&#x27;int&#x27;, dest=&#x27;numaddrs&#x27;)
parser.add_option(&#x27;-u&#x27;, &#x27;--used&#x27;, default=50, help=&#x27;percent of virtual address space to use&#x27;,
                action=&#x27;store&#x27;, type=&#x27;int&#x27;, dest=&#x27;used&#x27;)
parser.add_option(&#x27;-v&#x27;, &#x27;--verbose&#x27;, help=&#x27;verbose mode&#x27;,
                action=&#x27;store_true&#x27;, dest=&#x27;verbose&#x27;)
parser.add_option(&#x27;-c&#x27;, &#x27;--help&#x27;, help=SUPPRESS_HELP, action='store_true', dest='help')
```

```
(options, args) = parser.parse_args()
```

```
print &#x27;ARG seed&#x27;, options.seed
print &#x27;ARG address space size&#x27;, options.asize
print &#x27;ARG phys mem size&#x27;, options.psize
print &#x27;ARG page size&#x27;, options.pagesize
print &#x27;ARG verbose&#x27;, options.verbose
print &#x27;ARG addresses&#x27;, options.addresses
print &#x27;&#x27;
```

```
random.seed(options.seed)
```

```
asize = convert(options.asize)
psize = convert(options.psize)
pagesize = convert(options.pagesize)
addresses = str(options.addresses)
if psize &lt;= 1:
```

```

    print &#x27;Error: must specify a non-zero physical memory size.&#x27;
    exit(1)

if asize < 1:
    print &#x27;Error: must specify a non-zero address-space size.&#x27;
    exit(1)

if psize <= asize:
    print &#x27;Error: physical memory size must be GREATER than address space size (for this simulation)&#x27;
    exit(1)

if psize >= convert(&#x27;1g&#x27;) or asize >= convert(&#x27;1g&#x27;):
    print &#x27;Error: must use smaller sizes (less than 1 GB) for this simulation.&#x27;
    exit(1)

mustbemultipleof(asize, pagesize, &#x27;address space must be a multiple of the pagesize&#x27;)
mustbemultipleof(psize, pagesize, &#x27;physical memory must be a multiple of the pagesize&#x27;)

# print some useful info, like the darn page table
pages = psize / pagesize;
import array
used = array.array(&#x27;i&#x27;)
pt = array.array(&#x27;i&#x27;)
for i in range(0,pages):
    used.insert(i,0)
vpages = asize / pagesize

# now, assign some pages of the VA
vabits = int(math.log(float(asize))/math.log(2.0))
mustbepowerof2(vabits, asize, &#x27;address space must be a power of 2&#x27;)
pagebits = int(math.log(float(pagesize))/math.log(2.0))
mustbepowerof2(pagebits, pagesize, &#x27;page size must be a power of 2&#x27;)
vpnbits = vabits - pagebits
pagemask = (1 << pagebits) - 1

# import ctypes
# vpnmask = ctypes.c_uint32(~pagemask).value
vpnmask = 0xFFFFFFFF & ~pagemask
#if vpnmask2 != vpnmask:
#     print &#x27;ERROR&#x27;
#     exit(1)
# print &#x27;va:%d page:%d vpn:%d -- %08x %08x&#x27; % (vabits, pagebits, vpnbits, vpnmask, pagemask)

print &#x27;&#x27;
print &#x27;The format of the page table is simple:&#x27;
print &#x27;The high-order (left-most) bit is the VALID bit.&#x27;
print &#x27; If the bit is 1, the rest of the entry is the PFN.&#x27;
print &#x27; If the bit is 0, the page is not valid.&#x27;
print &#x27;Use verbose mode (-v) if you want to print the VPN # by&#x27;
print &#x27;each entry of the page table.&#x27;
print &#x27;&#x27;

print &#x27;Page Table (from entry 0 down to the max size)&#x27;
for v in range(0,vpages):
    done = 0
    while done == 0:
        if ((random.random() * 100.0) > (100.0 - float(options.used))):
            u = int(pages * random.random())
            if used[u] == 0:
                used[u] = 1
                done = 1
                # print &#x27;%d - %d&#x27; % (v, u)
                if options.verbose == True:
                    print &#x27; [%8d] &#x27; % v,
                else:
                    print &#x27; &#x27;,
                    print &#x27;0x%08x&#x27; % (0x80000000 | u)
                    pt.insert(v,u)
            else:
                # print &#x27;%d - not valid&#x27; % v

```

```

        if options.verbose == True:
            print &#x27; [%8d] &#x27; % v,
        else:
            print &#x27; &#x27;,
            print &#x27;0x%08x&#x27; % 0
            pt.insert(v,-1)
            done = 1
print &#x27;&#x27;

#
# now, need to generate virtual address trace
#

addrList = []
if addresses == &#x27;-1&#x27;:
    # need to generate addresses
    for i in range(0, options.num):
        n = int(ascii * random.random())
        addrList.append(n)
else:
    addrList = addresses.split(&#x27;,&#x27;)

print &#x27;Virtual Address Trace&#x27;
for vStr in addrList:
    # vaddr = int(ascii * random.random())
    vaddr = int(vStr)
    if options.solve == False:
        print &#x27; VA 0x%08x (decimal: %8d) --&gt; PA or invalid address?&#x27; % (vaddr, vaddr)
    else:
        paddr = 0
        # split vaddr into VPN | offset
        vpn = (vaddr & vpnmask) &gt;&gt; pagebits
        if pt[vpn] &lt; 0:
            print &#x27; VA 0x%08x (decimal: %8d) --&gt; Invalid (VPN %d not valid)&#x27; % (vaddr, vaddr, vpn)
        else:
            pfn = pt[vpn]
            offset = vaddr & pagemask
            paddr = (pfn &lt;&lt; pagebits) | offset
            print &#x27; VA 0x%08x (decimal: %8d) --&gt; %08x (decimal %8d) [VPN %d]&#x27; % (vaddr, vaddr, paddr)
print &#x27;&#x27;

if options.solve == False:
    print &#x27;For each virtual address, write down the physical address it translates to&#x27;
    print &#x27;OR write down that it is an out-of-bounds address (e.g., segfault).&#x27;
    print &#x27;&#x27;

```

File: paging-multilevel-translate.py

```
#!/usr/bin/env python

import sys
from optparse import OptionParser, SUPPRESS_HELP
import random
import math

def convert(size):
    length = len(size)
    lastchar = size[length-1]
    if (lastchar == '&#x27;k&#x27;') or (lastchar == '&#x27;K&#x27;'):
        m = 1024
        nsize = int(size[0:length-1]) * m
    elif (lastchar == '&#x27;m&#x27;') or (lastchar == '&#x27;M&#x27;'):
        m = 1024*1024
        nsize = int(size[0:length-1]) * m
    elif (lastchar == '&#x27;g&#x27;') or (lastchar == '&#x27;G&#x27;'):
        m = 1024*1024*1024
        nsize = int(size[0:length-1]) * m
    else:
        nsize = int(size)
    return nsize

def roundup(size):
    value = 1.0
    while value < size:
        value = value * 2.0
    return value

class OS:
    def __init__(self):
        # 4k phys memory (128 pages)
        self.pageSize = 32
        self.physPages = 128
        self.physMem = self.pageSize * self.physPages
        self.vaPages = 1024
        self.vaSize = self.pageSize * self.vaPages
        self.pteSize = 1
        self.pageBits = 5 # log of page size

        # os tracks
        self.usedPages = []
        self.usedPagesCount = 0
        self.maxPageCount = self.physMem / self.pageSize

        # no pages used (yet)
        for i in range(0, self.maxPageCount):
            self.usedPages.append(0)

        # set contents of memory to 0, too
        self.memory = []
        for i in range(0, self.physMem):
            self.memory.append(0)

        # associative array of pdr&#x27;s (indexed by PID)
        self.pdr = {}

        # mask is 11111 00000 00000 --&gt; 0111 1100 0000 0000
        self.PDE_MASK = 0x7c00
        self.PDE_SHIFT = 10

        # 00000 11111 00000 -&gt; 000 0011 1110 0000
        self.PTE_MASK = 0x03e0
        self.PTE_SHIFT = 5

        self.VPN_MASK = self.PDE_MASK | self.PTE_MASK
        self.VPN_SHIFT = self.PTE_SHIFT
```

```

# grabs the last five bits of a virtual address
self.OFFSET_MASK = 0x1f

def findFree(self):
    assert(self.usedPagesCount < self.maxPageCount)
    look = int(random.random() * self.maxPageCount)
    while self.usedPages[look] == 1:
        look = int(random.random() * self.maxPageCount)
    self.usedPagesCount = self.usedPagesCount + 1
    self.usedPages[look] = 1
    return look

def initPageDir(self, whichPage):
    whichByte = whichPage << self.pageBits
    for i in range(whichByte, whichByte + self.pageSize):
        self.memory[i] = 0x7f

def initPageTablePage(self, whichPage):
    self.initPageDir(whichPage)

def getPageTableEntry(self, virtualAddr, ptePage, printStuff):
    pteBits = (virtualAddr & self.PTE_MASK) >> self.PTE_SHIFT
    pteAddr = (ptePage << self.pageBits) | pteBits
    pte = self.memory[pteAddr]
    valid = (pte & 0x80) >> 7
    pfn = (pte & 0x7f)
    if printStuff == True:
        print "&#x27;  --&gt; pte index:0x%x [decimal %d] pte contents:0x%x (valid %d, pfn 0x%02x [decimal %d])" % \
            (pteBits, pteBits, pte, valid, pfn, pfn)
    return (valid, pfn, pteAddr)

def getPageDirEntry(self, pid, virtualAddr, printStuff):
    pageDir = self.pdbr[pid]
    pdeBits = (virtualAddr & self.PDE_MASK) >> self.PDE_SHIFT
    pdeAddr = (pageDir << self.pageBits) | pdeBits
    pde = self.memory[pdeAddr]
    valid = (pde & 0x80) >> 7
    ptPtr = (pde & 0x7f)
    if printStuff == True:
        print "&#x27;  --&gt; pde index:0x%x [decimal %d] pde contents:0x%x (valid %d, pfn 0x%02x [decimal %d])" % \
            (pdeBits, pdeBits, pde, valid, ptPtr, ptPtr)
    return (valid, ptPtr, pdeAddr)

def setPageTableEntry(self, pteAddr, physicalPage):
    self.memory[pteAddr] = 0x80 | physicalPage

def setPageDirEntry(self, pdeAddr, physicalPage):
    self.memory[pdeAddr] = 0x80 | physicalPage

def allocVirtualPage(self, pid, virtualPage, physicalPage):
    # make it into a virtual address, as everything uses this (and not VPN)
    virtualAddr = virtualPage << self.pageBits
    (valid, ptPtr, pdeAddr) = self.getPageDirEntry(pid, virtualAddr, False)
    if valid == 0:
        # must allocate a page of the page table now, and have the PD point to it
        assert(ptPtr == 127)
        ptePage = self.findFree()
        self.setPageDirEntry(pdeAddr, ptePage)
        self.initPageTablePage(ptePage)
    else:
        # otherwise, just extract page number of page table page
        ptePage = ptPtr
    # now, look up page table entry too, and mark it valid and fill in translation
    (valid, pfn, pteAddr) = self.getPageTableEntry(virtualAddr, ptePage, False)
    assert(valid == 0)
    assert(pfn == 127)
    self.setPageTableEntry(pteAddr, physicalPage)

# -2 -&gt; PTE fault, -1 means PDE fault
def translate(self, pid, virtualAddr):

```

```

(valid, ptPtr, pdeAddr) = self.getPageDirEntry(pid, virtualAddr, True)
if valid == 1:
    ptePage = ptPtr
    (valid, pfn, pteAddr) = self.getPageTableEntry(virtualAddr, ptePage, True)
    if valid == 1:
        offset = (virtualAddr & self.OFFSET_MASK)
        paddr = (pfn << self.pageBits) | offset
        print &#x27;    --&gt; pfn: %02x  offset: %x&#x27; % (pfn, offset)
        return paddr
    else:
        return -2
return -1

def fillPage(self, whichPage):
    for j in range(0, self.pageSize):
        self.memory[(whichPage * self.pageSize) + j] = int(random.random() * 31)

def procAlloc(self, pid, numPages):
    # need a PDBR: find one somewhere in memory
    pageDir = self.findFree()
    # print &#x27; **ALLOCATE** page dir&#x27;; pageDir
    self.pdbr[pid] = pageDir
    self.initPageDir(pageDir)

    used = {}
    for vp in range(0, self.vaPages):
        used[vp] = 0
    allocatedVPs = []

    for vp in range(0, numPages):
        vp = int(random.random() * self.vaPages)
        while used[vp] == 1:
            vp = int(random.random() * self.vaPages)
        assert(used[vp] == 0)
        used[vp] = 1
        allocatedVPs.append(vp)
        pp = self.findFree()
        # print &#x27; **ALLOCATE** page&#x27;; pp
        # print &#x27; trying to map vp:%08x to pp:%08x&#x27; % (vp, pp)
        self.allocVirtualPage(pid, vp, pp)
        self.fillPage(pp)
    return allocatedVPs

def dumpPage(self, whichPage):
    i = whichPage
    for j in range(0, self.pageSize):
        print self.memory[(i * self.pageSize) + j],
    print &#x27;&#x27;

def memoryDump(self):
    for i in range(0, self.physMem / self.pageSize):
        print &#x27;page %3d:&#x27; % i,
        for j in range(0, self.pageSize):
            print &#x27;%02x&#x27; % self.memory[(i * self.pageSize) + j],
        print &#x27;&#x27;

def getPDBR(self, pid):
    return self.pdbr[pid]

def getValue(self, addr):
    return self.memory[addr]

# allocate some processes in memory
# allocate some multi-level page tables in memory
# make a bit of a mystery:
# can examine PDBR (PFN of current proc&#x27;s page directory)
# can examine contents of any page
# fill pages with values too
# ask: when given
#   LOAD VA, R1

```

what will final value will be loaded into R1?

```
#
# main program
#
```

```
parser = OptionParser()
parser.add_option('--s', &#x27;&#x27;, &#x27;&#x27;--seed&#x27;&#x27;, default=0, help=&#x27;&#x27;the random seed&#x27;&#x27;, action=&#x27;&#x27;
parser.add_option('--a', &#x27;&#x27;, &#x27;&#x27;--allocated&#x27;&#x27;, default=64, help=&#x27;&#x27;number of virtual pages allocated&#x27;&#x27;,
                action=&#x27;&#x27;store&#x27;&#x27;, type=&#x27;&#x27;int&#x27;&#x27;, dest=&#x27;&#x27;allocated&#x27;&#x27;
parser.add_option('--n', &#x27;&#x27;, &#x27;&#x27;--addresses&#x27;&#x27;, default=10, help=&#x27;&#x27;number of virtual addresses to allocate&#x27;&#x27;,
                action=&#x27;&#x27;store&#x27;&#x27;, type=&#x27;&#x27;int&#x27;&#x27;, dest=&#x27;&#x27;num&#x27;&#x27;)
parser.add_option('--c', &#x27;&#x27;, &#x27;&#x27;--solve&#x27;&#x27;, help=SUPPRESS_HELP, action=&#x27;&#x27;store_false&#x27;&#x27;, default=True)
```

```
(options, args) = parser.parse_args()
```

```
print &#x27;&#x27;ARG seed&#x27;&#x27;, options.seed
print &#x27;&#x27;ARG allocated&#x27;&#x27;, options.allocated
print &#x27;&#x27;ARG num&#x27;&#x27;, options.num
print &#x27;&#x27;&#x27;
```

```
random.seed(options.seed)
```

```
# do the work now
os = OS()
used = os.procAlloc(1, options.allocated)
```

```
os.memoryDump()
```

```
print &#x27;&#x27;\nPDBR:&#x27;&#x27;, os.getPDBR(1), &#x27;&#x27; (decimal) [This means the page directory is held in this page]\
```

```
for i in range(0, options.num):
    if (random.random() * 100) > 50.0 or i >= len(used):
        vaddr = int(random.random() * 1024 * 32)
    else:
        vaddr = (used[i] & 5) | int(random.random() * 32)
    if options.solve == True:
        print &#x27;&#x27;Virtual Address %04x:&#x27;&#x27; % vaddr
        r = os.translate(1, vaddr)
        if r > -1:
            print &#x27;&#x27;    --&#x27;&#x27; Translates to Physical Address 0x%03x --&#x27;&#x27; Value: %02x&#x27;&#x27; % (r, os.getPDBR(1))
        elif r == -1:
            print &#x27;&#x27;    --&#x27;&#x27; Fault (page directory entry not valid)&#x27;&#x27;
        else:
            print &#x27;&#x27;    --&#x27;&#x27; Fault (page table entry not valid)&#x27;&#x27;
    else:
        print &#x27;&#x27;Virtual Address %04x: Translates to what Physical Address (and fetches what value)? Or Fault?
```

```
print &#x27;&#x27;&#x27;
```

```
exit(0)
```